Joshua Dalton
IT 505

1. “Testing” is often used alongside “debugging” by programmers. Distinguish between the two and explain their relationship.

Testing is checking if a program works and debugging is finding and fixing problems in the code itself. They work together in that testing helps identify the issues and debugging fixes them.

2. What role does static type checking play in testing TypeScript programs? Can testing stop once a program passes that type check without generating errors?

Static type checking checks for errors in types before the program runs. Testing shouldn't stop after this because there could still be other errors that type checking doesn't find.

3. Describe the process of code tracing and explain how it might be useful in the context of texting and debugging. How might function and method invocations be handled in this process and why? What tool is most useful for code tracing and why?

Code tracing is “following a program step by step” to see how it runs and how things change. When a function or method is called, you can trace through its code before returning to where it was called, helping find where errors occur. A debugger is useful for this because it lets you go through the code one line at a time and check values.

4. What role does a design play in the programming process? Does that design then have a role to play in the testing process? If so, what is its role there?

The role of design in programming is to help plan out how a program should work before actually writing the code. It’s also useful during testing because you can compare how the program works vs. how it was originally designed to work.

5. The Deno Standard Library contains an assert() function. What does this function accomplish? How might that behavior be useful for testing and debugging purposes? What line of code must appear in your program before you can actually use the assert() Function?

The assert() function checks if a condition is true, otherwise throws an error if it’s false. This is useful for testing and debugging because it helps find when something isn't working correctly, like if you wanted to check if a function that added two numbers returned the correct number.

Line of code needed: import { assert } from "jsr:@std/assert";

6. Deno also makes an assert() method available within the console object. Compare and contrast this version to the one described above in the Standard Library.

Both versions check if a condition is true or false. The difference is that the Standard Library version throws an error if the condition is false, while console.assert() just prints an error message.

Joshua Dalton
IT 505

7. What is a stack trace (or traceback) and how might it be useful for testing and debugging purposes? How can you generate a stack trace explicitly at a point of your choosing within the code? Under what circumstances are you likely to encounter a stack trace that has been automatically generated for you? How might your decision whether to use function statements or function expressions impact the usefulness of a stack Trace?

A stack trace shows the order of function calls that led to a certain point or error in a program. It's useful for debugging because it helps in finding where an error occurred. You can manually generate one using console.trace(), but they are also automatically generated when an error occurs. Using named functions make stack traces easier to understand because the function names will appear in the trace.

8. Deno supports console.count() and console.countReset(). What do these methods do and how might they be useful in testing and debugging?

console.count() method keeps track of how many times a part of the code runs. console.countReset() resets the count back to 0. These are useful for testing and debugging because they help check how many times a function or section of code is executed.

9. How might you use console.log() in the testing and debugging process? When debugging it is common to examine the value of a variable (call it myVar for this example) by placing the variable inside a set of curly braces when passing it as an argument to console.log() (for example, console.log({myVar})). What does this accomplish and why is it useful?

console.log() can be used to print values and messages to help find errors in a program. Using console.log({myVar}) prints both the variable's name and its value instead of just the value itself. This is useful because it makes it easier to identify which variable is being checked during debugging.

10. What is a debugger? What types of functionality does a typical debugger offer? Does Deno offer one? Does VS Code? Do you feel a debugger should be the first tool for which you reach when debugging or a last resort? Explain.

A debugger is a tool that helps find errors in a program by letting you pause the code, go through it step by step, and check variable values. Both Deno and VS Code support debugging. I think a debugger should be used when simpler methods aren't enough because smaller errors can usually be found without needing one.

11. Compare and contrast unit testing with integration testing. In what order are they most commonly completed? List some other types of testing that typically follow them in the process.

Unit testing = Checks individual parts of a program to make sure they work correctly.
Integration testing = Checks if different parts of a program work together correctly.
Unit testing is usually done first, followed by integration testing. Other types:

- **System testing**
- **Performance testing**

Joshua Dalton
IT 505

- **Acceptance testing**

12. What is a test framework? When might a test framework be useful? Name two test frameworks that are compatible with TypeScript.

A test framework is a tool that organizes and runs tests for a program. It's useful when testing bigger programs or when you need to run multiple tests automatically. List:

- **Deno's built-in test framework**
- **Jest**

13. What is a software test plan and how might one be used?

A software test plan explains what needs to be tested in a program, how to test it, and what should happen. It helps make sure everything gets tested.

14. What is concurrency in the context of computing? Is it supported by TypeScript? If so, through what mechanism(s)?

Concurrency is when a program can work on multiple tasks at once instead of completing one before starting another. Yes, TS supports concurrency through asynchronous programming using promises, async/await, and mechanisms like "Web Workers".

15. What is asynchronous programming? Compare and contrast it with synchronous programming. (Hint: A good start on answering this can be found in the overview above.)

It lets a program start a task and continue running other code while waiting for that task to finish. Synchronous programming runs code in order. Both are used to execute code, but asynchronous programming is useful when tasks take time to complete, like waiting for data from an API.

16. How are callbacks typically used in asynchronous programming? What is a nested callback and why are nested callbacks potentially problematic for us as programmers? What is the most common alternative to nested callbacks?

In asynchronous programming, callbacks are used to run certain code after a task finishes or when an event occurs without stopping the rest of the program while it waits. For example, a callback could run when a user clicks a button. A nested callback is when a callback is inside another callback. This can make code harder to read, understand, and debug. The most common alternative to nested callbacks is called promises.

17. What is a promise and how are promises typically used in asynchronous programming?

An object that represents the result of an asynchronous task that can finish later. Promises are used to handle tasks that take time to complete.

18. What are the three possible states for a promise and what does each indicate? What does it mean for a promise to be called settled?

Joshua Dalton
IT 505

- **Pending = The promise is still waiting for the task to finish.**
- **Fulfilled = The task completed successfully.**
- **Rejected = The task failed or an error occurred.**

A promise is considered settled when it's been fulfilled or rejected.

19. What is the importance of a promise's then() method and how is it relevant to a settled promise? What's important to keep in mind about the argument you pass to the then() Method?

It's used to run code when a promise is fulfilled. It can also be used after a promise has already settled. It's important to remember the argument passed to then() should be a callback function that will run when the promise is fulfilled, instead of calling the function immediately.

20. Explain the process of promise chaining and how it relates to the idea of a pipeline. How does it relate to method chaining? When might it be useful?

"**Promise chaining" is when multiple then() methods are connected so the result of one can be passed to the next. It works like a "pipeline" because each step uses the result from the previous step. It's similar to method chaining because multiple methods are called one after another. This is useful when asynchronous tasks need to be completed in a specific order.**

21. What is an immediately settled promise and how might one be used? Why are immediately settled promises of particular relevance to promise chaining?

An immediately settled promise is a promise that is already fulfilled or rejected when it's created. They can be created using Promise.resolve() or Promise.reject(). They are useful in promise chaining because they let a value (or error) pass into the chain without waiting for an asynchronous task to finish.

22. What is the importance of a promise's catch() method and how is it relevant to a settled promise? What role does the catch() method typically play in a promise chain?

catch() is used to handle errors when a promise is rejected. It runs when a promise settles as rejected and it helps prevent errors from going unhandled. In a promise chain, catch() is usually placed near the end to handle errors from earlier steps in said chain.

23. What is the importance of a promise's finally() method? What role does the finally() method typically play in a promise chain?

finally() runs code after a promise has settled, whether it was fulfilled or rejected. It's usually placed at the end of a promise chain to do cleanup tasks that need to happen regardless of whether or not the promise succeeded or failed.

24. For what purpose would you use the Promise.all() method? What happens if one of the promises passed into Promise.all() is rejected? What additional information is available if you instead use Promise.allSettled()?

Joshua Dalton
IT 505

Promise.all() is used to wait for multiple promises to finish and get their results. If one promise is rejected, Promise.all() will reject as well. Promise.allSettled() waits for all promises to settle and shows whether each one was fulfilled or rejected, along with its result or reason for the failure.

25. Is it possible to define your own custom promises? If so, are you more likely to generate your own promises or utilize promises returned by the code of others? Why?

Yes, by using the Promise constructor. You are more likely to use promises returned by existing functions or APIs because many asynchronous operations already provide them, making it easier to write code without creating your own.

26. What is an executor function in the context of a promise? What are the rules for writing an executor function?

An executor function is a function that runs when a new promise is created. Rules: It takes two functions as parameters, resolve and reject, which are used to indicate whether the task succeeded or failed. It should call resolve() when successful or reject() when an error occurs.

27. What is an async function? What is the role of the await operator within an async function? Are there any Deno-specific considerations relevant to the use of the await keyword? Where does the async keyword appear relative to the function keyword (when present)? Where does it appear in an arrow function (which doesn't have a function keyword)? What does an async function generate as its return value?

An async function is a function that allows asynchronous code to be written in a way that looks synchronous. The await keyword pauses the function until a promise is settled. Deno also allows await to be used outside of async functions at the top level of a module. The async keyword goes before the function keyword or before the parameters in an arrow function. An async function returns a promise.